XCEL UX DASHBOARD · FOR DESIGNERS

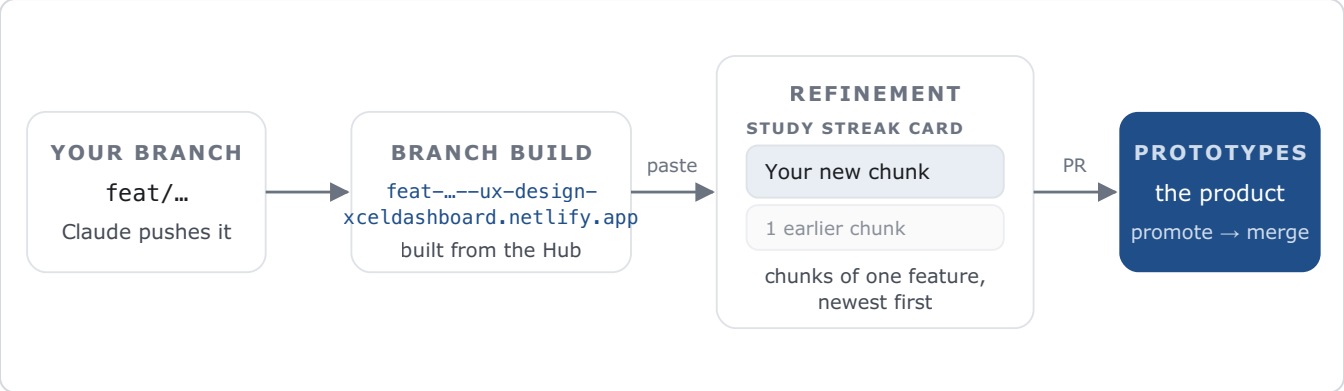
Contributing to the dashboard

How to get your work in front of the team and then in front of stakeholders — from getting the project onto your computer to seeing your change land in Prototypes. Everything happens in the **Claude desktop app**: each step is a sentence you say to Claude. Nothing is typed into a terminal, and github.com is only opened once, to accept the invitation.

Last updated 26 October 2026 · Owner: Jillienne Moreno · Applies to `jillmoreno/xcel-ux-dashboard`

PART 1 · QUICK WALKTHROUGH

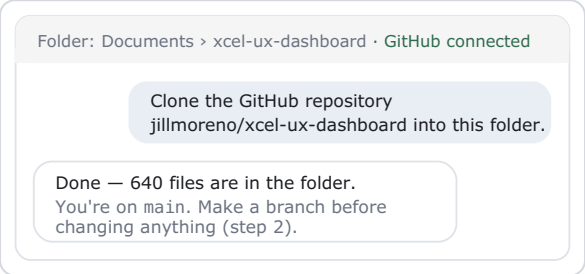
The loop, in one picture



Left to right: you make a branch and Claude pushes your work to it, Netlify builds it at its own address, that address goes into Refinement where it joins the other chunks of the same feature, and a PR plus the promote step lands it in Prototypes. Everyone with the Hub link — the team and stakeholders alike — sees a Refinement row the moment it is added. Everything below is one of those boxes in detail.

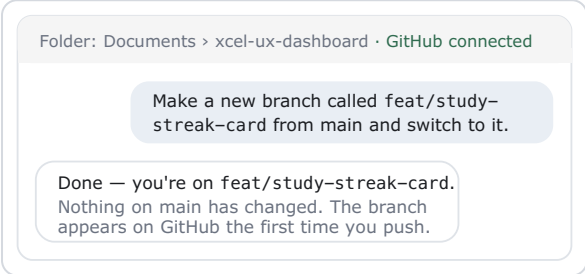
Eight steps

1 Get the project onto your computer



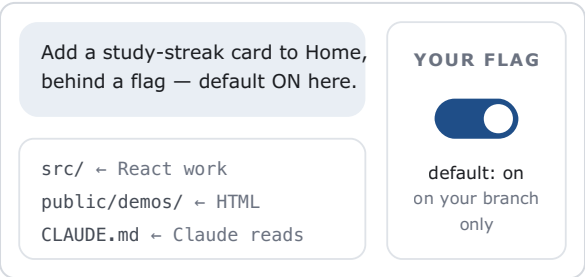
Once. Make an empty folder, open the Claude desktop app, connect that folder, and say *"Clone the GitHub repository jillmoreno/xcel-ux-dashboard into this folder."* Claude brings it down. You do **not** run the site locally — your preview is the branch build in step 5.

2 Make a branch



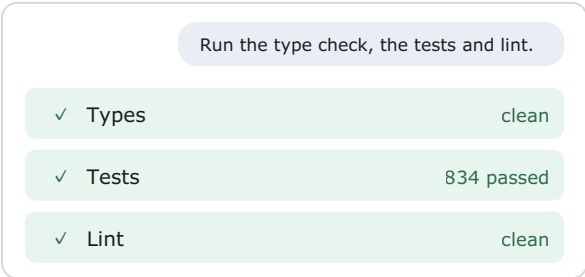
Tell Claude *"Make a new branch called feat/your-thing from main and switch to it."* Name it `feat/` plus two or three words, lowercase, joined with dashes. A branch is your own copy to work in — nothing on it changes what anyone else sees until it is merged.

3 Build it with Claude



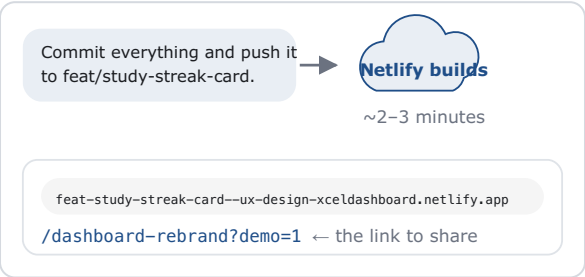
In the Claude desktop app, describe the work. Say which kind it is — a React surface (`src/`) or an HTML exploration (`public/demos/`) — and ask for a feature flag with its default **on** on your branch. Every so often: *"Commit what we have so far."*

4 Ask Claude to run the checks



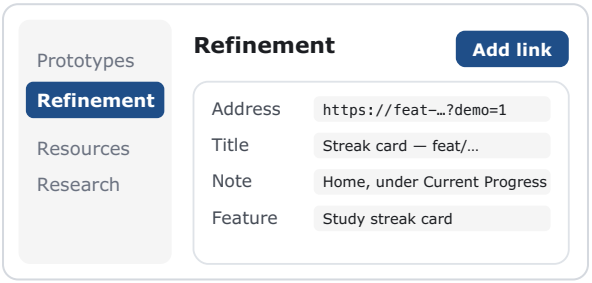
Before every publish, say *"Run the type check, the tests and lint."* Claude runs them and reports. A failing test is usually a recorded decision talking to you — ask Claude what it protects. Touched `public/prototypes/` ? Add *"and the smoke suites."*

5 Publish — that is the deploy



Say *"Commit everything and push it to feat/your-thing."* Claude sends it to GitHub; Netlify builds your branch as its own site. Share `…/dashboard-rebrand?demo=1` so reviewers see your committed defaults. **Every pushed branch gets an address** — anyone with the link and the site password can open it.

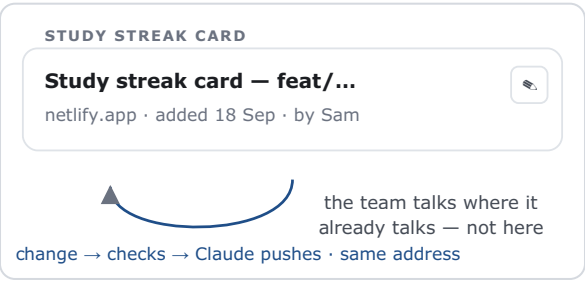
6 Add it to Refinement



Refinement → **Add link**. Paste the address, title it with your branch name, say where to look, and name the **feature** this is a chunk of. Everyone with the Hub link — the team and stakeholders alike — sees it the moment you click Add.

Much faster — it covers steps 4 and 5 as well: in your Claude chat, say `/share-branch`. It runs the checks, commits, pushes, waits for Netlify to build your branch, works out the address, drafts the title and note, asks what feature this is a chunk of, and gives you a link that opens this form already filled in — you check it and click Add. Then it offers to cut your next branch. (`/promote-to-refinement` is the old name and still works.)

7 Discuss and iterate



The team opens the row and talks. You change with Claude, ask for the checks, ask Claude to push again — the address stays the same, so the row stays right. **Edit** the row if its title or note should change.

8 PR → promote → merge → retire



Tell Claude *"Open a pull request from feat/your-thing to main and add Jillienne as reviewer."* She runs `promote-to-prototype` on your branch — each flag your branch changed is kept on or set back to `main`'s value — then merges. The Hub rebuilds. Then say `/retire-from-refinement` to take the row off — it records

what was decided before it goes. When the feature is built and engineering needs to read it, that is `/dev-handoff-notes` — Part 6 of the day-to-day guide — written on your branch before the PR, so it merges with the work.

Four rules that cover most mistakes

NEVER PUSH TO MAIN

Every push to `main` rebuilds the Hub for everyone. Branch, then PR.

PROMOTION IS A FLAG, NOT A TILE

Prototypes holds one row per product surface — the dashboard, Compass — and a test guards the list. Your work reaches a row by a flag default flipping on `main`.

REVIEW HTML GOES IN PUBLIC/DEMOS/

Both folders work on your branch build. `public/demos/` is for work in review and is meant to be short-lived; `public/prototypes/` is for a document that is becoming a permanent row. Only the second needs a row in the project list.

EVERY BRANCH GETS AN ADDRESS

Anyone with the link and the site password can open it — including work you are not ready to show. Nothing confidential on a branch.

CONTENTS

1. The big picture
2. What you need before you start
3. Two kinds of work, two homes
4. Feature flags — the one rule
5. Things not to do
6. Then: the day-to-day guide

01 The big picture

The dashboard is **one codebase deployed to one website** — the Hub, behind one site password that designers, developers and stakeholders are all given. Every `git push` to `main` rebuilds it. (Until 2026-10-15 there was a second, trimmed site for stakeholders; it is retired.)

GROUP IN THE RAIL	SECTIONS	WHAT THEY ARE
Demo	Prototypes	The product as it stands — one row per surface, every row `main`'s build.
Design & Research	Refinement · Research · Resources · PRDs · Explorations	Four boards authored on the page, then the hand-authored documents — outside products and ideas rebuilt on our tokens.
Dev Handoff	Development · Done · Archive · QA Notes	What engineering is building, what shipped, what was removed, and design-vs-build findings.

Nothing in the rail is locked: the site password is the only door. The old *Designers* group (Design · Exploration behind a second gate) went on 2026-10-15.

Two sections do the work of getting a design in front of people, and they are different acts:

- **Refinement** is the review inbox. It is a list of links you add *on the page* — no code, no commit. A row points at your branch's own build (or an HTML file on it), and it is **one chunk of work**: rows group under the *feature* they are chunks of, newest first, so a run of small branches reads as one piece of work with a history you can still click through. Everyone with the Hub link sees every row.
- **Prototypes** is the product as it stands — the live React build at the flag defaults committed on `main`. Only a merge to `main` changes it, and only Jillienne decides what merges into that baseline.

So the loop is:

branch → `/share-branch` → everyone with the Hub link sees it → discuss & iterate → PR → promote → merge → retire the row → next chunk

The rest of this guide walks each of those, in order, with the exact commands and the exact fields.

02 What you need before you start

- **A GitHub account, with access to the project.** Jillienne invites you; GitHub emails you; click *Accept*. To check it worked, open `github.com/jillmoreno/xcel-ux-dashboard` in your browser — if you see a list of files rather than a 404, you are in.
- **The Claude desktop app, with GitHub connected.** In Claude: *Settings* → *Connectors* → *GitHub* → *Connect*, sign in to GitHub when it asks, and allow it. This is what lets Claude bring the project down to your computer and send your changes back up. Without it Claude can edit files but cannot talk to GitHub.
- **An empty folder on your computer** for the project — for example *Documents › xcel-ux-dashboard*. Claude fills it in step 1.
- **The full site's address.** Ask Jillienne.

> **YOU DO NOT NEED**
>
> Terminal, Node, git, GitHub Desktop, or a Netlify account. Anything that would normally need a typed command, Claude does when you ask it in plain English. You never have to know what the command was.

03 The words you will see

Six of them, over and over. You never type a git command — Claude does that — but you will read these in what it tells you.

WORD	PLAIN MEANING
`main`	The one branch that IS the product. Only Jillienne changes it.
branch	Your own copy of the code, named like `feat/compass-week-view`. Everything you do happens on a branch.
commit	A saved snapshot of your branch with a short note saying what changed.
push	Sending your commits to GitHub. Netlify then builds them and the team can see them.
rebase	Moving your branch so it starts from today's `main` instead of the day you created it. You will do this every few days; the day-to-day guide has the steps.
merge	Jillienne putting your branch into `main`. She calls this "ship to main".

> THE ONE RULE UNDER EVERYTHING
>
> **You work on your branch, Jillienne works on `main`.** If you ever find yourself on `main`, stop and tell her.

04 Two kinds of work, two homes

Where files go depends on what you are making, and picking wrong is the most common mistake in this repo.

YOU ARE MAKING	IT LIVES IN	REVIEWERS OPEN	PROMOTED BY
A React surface of the product	`src/` — a page in the shell, a card, a panel, a variant of an existing block. Behind a feature flag.	`<branch>--…netlify.app/dashboard-rebrand?demo=1`	Flipping the flag's default on `main` (the promote skill).
A standalone HTML exploration	`public/demos/<your-file>.html` — a hand-authored page answering a brief, not wired into the app.	`<branch>--…netlify.app/demos/<your-file>.html`	Moving it to `public/prototypes/` with a row in `prototypeFeatures.ts` if it is a finished exploration, or rebuilding it in `src/` if it became a product surface.

> **WHY `PUBLIC/DEMOS/` AND NOT `PUBLIC/PROTOTYPES/`**
>
> `public/prototypes/` holds the finished standalone explorations — each one has a row in the feature list and its own smoke assertions, and the user-test build 404s the whole folder at the edge. `public/demos/` is the folder for HTML in review, and it is deliberately left open everywhere. Nothing should live there long, and nothing there gets a row in the feature list — the Refinement panel is its listing.

05 Feature flags — the one rule

Every change to the product starts **invisible to Prototypes**: behind a flag whose default on `main` is off (or set to the existing variant). That is what lets many designers work on the same product without anyone's half-finished idea appearing in front of stakeholders.

On **your branch**, you set that flag's default *on*. That is what makes your branch build show your work at `?demo=1`. It also means the promote skill can see, mechanically, exactly which defaults your branch changed — the list it asks Jillienne about in step 8 is the diff of the flag catalog between your branch and `main`.

- Flags live in `src/context/FeatureFlagContext.tsx`, in `FEATURE_FLAGS`. Ask Claude to add yours there and to add it to the `/dashboard-rebrand` panel scope so reviewers can toggle it.

- Reviewers can flip flags live on your build: hover the prototype bar and find the robot icon, which opens the flag sheet. In demo mode those flips are temporary and reset on exit.
- Do not promote by adding a row to Prototypes. That section holds one row per product surface — all the same build on `main` — and a test compares it in both directions, so an unexpected row fails the suite.

06 Things not to do

- **Don't push to `main`.** Every push to `main` rebuilds the Hub, which stakeholders are looking at. Always be on a branch before asking Claude to push — step 2 — and get to `main` only through a pull request.
- **Don't add a row to Prototypes** or change a row's `category` to `'prototype'`. Promotion is a flag default, never a tile move.
- **Don't put review HTML in `public/prototypes/`.** That folder is the finished standalone explorations, each with a row in the feature list and its own smoke assertions. Work in review goes in `public/demos/`, which exists for exactly that and gets no row.
- **Don't change a flag's default on `main` yourself.** That is the promote step, and it is Jillienne's call.
- **Send your branch build, never the main dashboard.** Your branch address shows your work; the main address shows what is already decided, and everything else the team is doing.
- **Don't invent data.** A number, a URL, a lesson count or a fee that XCEL has not published is flagged as `INVENTED` in the code and named as such on screen. `CLAUDE.md` records why, several times over.
- **Don't draft the Quick Summary** on a dev-handoff component — ask Jillienne for it. The `dev-handoff-notes` skill says the same.

07 Then: the day-to-day guide

That is the setup done. Everything after your first day — rebasing, building a change, sharing it on Refinement, promoting a surface, writing the dev handoff, the ready-made routines, and what to say when something breaks — is the other guide:

Designer Workflow on the Hub →

The two used to describe the same loop at two lengths, which is why they felt repetitive. Since 15 October this one is **once, on your first day**, and that one is **every day after it**. The glossary and the troubleshooting table live there now, not here.

Part of the XCEL UX Dashboard. The reasoning behind every decision described here is in the repo's `CLAUDE.md`; the one-time site setup is in `README.md`. The day-to-day guide is Designer Workflow on the Hub; stakeholders get a shorter companion page, How to read this dashboard. All three sit on the Hub under **Team → Hub Resources**.